Week 2 Day 2 — Raw Python vs. LangChain Agent

Comparison write-up and annotated reasoning trace

What changed going from raw Python to LangChain

Aspect	Day 1 — Raw Python	Day 2 — LangChain
Tool calling	Hand-written TOOL_CALL:/INPUT: text format, parsed with regex	@tool decorator; model's function-calling handles parsing
Agent loop	Manual for-loop with max_iterations guardrail	AgentExecutor runs reason/act/observe internally
Memory	scratchpad dict appended to messages list by hand	RunnableWithMessageHistory keyed by session_id
Error handling	try/except inside execute_tool(), returns "ERROR: ..." string	handle_tool_error=True catches exceptions, feeds them back as an Observation
Structured output	Not implemented — final answer is plain text	with_structured_output(PydanticModel) returns a typed object
Visibility	Every prompt string and parsing step is visible and editable	Prompt construction and exception-to-message mapping are hidden inside the framework

LCEL Pipe Syntax (|) Under the Hood: The pipe operator composes components into a RunnableSequence where each component's output becomes the next component's input, handling type conversion automatically (dict → prompt → LLM → string).

Tools Implemented with @tool Decorator

Tool	Description	Data Source
calculator(expression)	Performs arithmetic calculations	In-memory (reused from Day 1)
get_weather(city)	Provides the temperature and weather condition for a given city	FAKE_WEATHER_DB (reused from Day 1)
search_products(query)	Searches for products and returns their name, price, and category	**CSV file (NEW - External Data Source)**

Tool docstrings are used as part of the prompt by giving the LLM clear information about each tool, including what it does and when it should be used. The framework automatically uses these descriptions to help the model select the appropriate tool and provide the required parameters.

Annotated reasoning trace

Query: "Look up the weather in Lahore and Tokyo and tell me which is warmer."

Step	Stage	What happened
1	Reason -> Act	Model decided to call get_weather(city="Lahore")
2	Observe	Tool returned "38C, Sunny"
3	Reason -> Act	Model decided to call get_weather(city="Tokyo")
4	Observe	Tool returned "27C, Clear"
5	Reason -> Final answer	Model compared 38C vs 27C and answered that Lahore is warmer

What's similar vs. what's now hidden

- Similar: the underlying loop is still reason -> act -> observe -> repeat, and the same two tool calls happen in the same order as Day 1's raw log.

- Hidden now: Day 1 printed the literal prompt text sent to Gemini and the raw TOOL_CALL:/INPUT: string parsed with regex. AgentExecutor's verbose=True shows what was called and returned, but not the exact request/response text or the parsing logic underneath.
- Hidden now: the exact prompt LangChain assembles for function-calling, and how a raised Python exception gets mapped into the text the model actually reads as an Observation.

Memory Test - 3-Turn Conversation

Turn	Query	Result
1	"Find the price of a Laptop"	Returns $1200
2	"Now compare it to the Smartphone"	Remembers the Laptop and compares it with the Smartphone at $800
3	"Which one should I recommend to a budget-conscious client?"	Recommends the Smartphone as the more budget-friendly option

The agent successfully maintained context throughout all three turns without needing to re-query previously mentioned information.

What LangChain made easier vs. what felt like "magic"

LangChain removed the need to hand-write the tool-call prompt format, the regex-based response parser, and the reason/act/observe loop itself — AgentExecutor, the @tool decorator, and with_structured_output cover all of that, and RunnableWithMessageHistory replaced the hand-rolled scratchpad dict for multi-turn memory. The trade-off is reduced visibility: the exact prompt built for tool-calling and the exception-to-message mapping happen inside the framework, so a wrong tool call is faster to fix here than in raw Python but harder to trace back to a root cause, since the intermediate text is no longer something the code prints or edits directly.